

Lineare Filter

Mittelungsfiler werden verwendet um Bildeigenschaften zu verändern (Rauschen, Kanten, Schärfe). Bei der Berechnung der Filter wird ein Kernel / eine Maske über das Bild geschoben und die Pixelwerte von Bild und Maske werden miteinander multipliziert und danach gemittelt. Je nach Wahl der Maske können hier unterschiedliche Effekte erzielt werden. In der Vorlesung haben Sie einige Filter schon kennengelernt und mit ImageJ ausprobiert. In dieser Übung stellen wir Ihnen nochmals die wichtigsten Filter vor. Zum Schluss betrachten wir dann welche Auswirkung die Form der Maske auf das Filterergebnis hat.

```
In [1]: # package import
import numpy as np
import matplotlib.pyplot as plt
import cv2
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from modules import plotting
import skimage
```

```
In [2]: # Load example Images
img_air = cv2.imread(r'images/airfield02g.png', cv2.IMREAD_GRAYSCALE)
img_bricks = cv2.imread(r'images/bricks.jpg', cv2.IMREAD_GRAYSCALE)
img_test = cv2.imread(r'images/testpattern.bmp', cv2.IMREAD_GRAYSCALE)
img_road = cv2.imread(r'images/road2.jpg', cv2.IMREAD_GRAYSCALE)
img_brickwall = cv2.imread(r'images/brickwall.bmp', cv2.IMREAD_GRAYSCALE)
img_houses = cv2.imread(r'images/houses.bmp', cv2.IMREAD_GRAYSCALE)
```

Bild Auswählen und für die weitere Filterung verwenden:

```
In [3]: # select image to analyze
image = img_houses
# display original image
plotting.displayImagePlotly(image, title='Original Image')
```

Original Image



Datentypen

Einige Filteroperationen benötigen den Bilddatentyp `float32`, damit die Berechnungen korrekt durchgeführt werden können. Deshalb wird das Bild vor der Filterung in diesen Datentyp konvertiert.

Hinweis Nach der Filteroperation muss das Bild dann wieder in ein `uint8` Bild umgewandelt werden. Hier gibt es 2 Möglichkeiten:\♦

- `np.clip(image, 0, 255)` : Hier werden alle Pixel <0 und >255 hart abgeschnitten. Pixelwerte können hierbei sättigen und gehen verloren
- `cv2.normalize(image, None, 0, 255, cv2.NORM_MINMAX)` : Hier wird das Bild so skaliert, sodass dessen Maximum / Minimum auf den Wert 255 bzw. 0 gezogen wird.

```
In [4]: image_f = image.astype('float32')
```

Mittelungsfiler

Hier werden die Mittelungsfiler vorgestellt. Wenn möglich wird die Filteroperation mit der OpenCV Funktion `cv2.filter2D` durchgeführt.

Die hier berechneten Filterfunktionen werden mit der OpenCV Funktion `filter2D` berechnet. Bei dieser Funktion kann man eine selbst erstellte Filtermatrix übergeben. Sowohl OpenCV als auch Skimage bieten für viele Funktionen schon vorprogrammierte Filter an, die einem das definieren der Filtermatrix abnehmen

Mittelungsfiler: `cv2.blur` Gaussfilter: `cv2.GaussianBlur`

```
In [5]: ### Mittelungsfiler
        ## Blurring / Gaussian Blur

        kernelsize = 5
        kernel = np.ones((kernelsize, kernelsize), np.float64) / (kernelsize * kernelsize)
```

```

# apply blur
blur = cv2.filter2D(image_f, -1, kernel)

#####Note #####
# This filter call is identical to the cv2-Filter
# cv2.blur(image_f, (kernelsize, kernelsize))

# transform image back to uint8
blur = np.clip(blur, 0, 255).astype('uint8')

print(f'Mittelungsfiler kernelsize: {kernelsize}')
print(kernel)
plotting.displayImagePlotly(blur, title=f'Blurred Image Kernelsize: {kernelsize}')

```

Mittelungsfiler kernelsize: 5

```

[[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]

```

Blurred Image Kernelsize: 5



In [6]: # gaussian blur

```

kernelsize = 5
sigma = 3

# create Gaussian Kernel
struct = cv2.getGaussianKernel(kernelsize, sigma)
kernel = struct @ struct.T

# Gaussfilter anwenden
gaussian = cv2.filter2D(image_f, -1, kernel)
#####Note #####
# This filter call is identical to the cv2-Filter
# cv2.GaussianBlur(image_f, (kernelsize, kernelsize), 2)

```

```
# transform image back to uint8
gaussian = np.clip(gaussian, 0, 255).astype('uint8')

print(f'Gaussformiger Kernel fernelsize:{kernelsize}, sigma:{sigma}')
print(kernel)
plotting.displayImagePlotly(gaussian, title='Gaussian Blurred Image')
```

```
Gaussformiger Kernel fernelsize:5, sigma:3
[[0.0317564  0.03751576 0.03965895 0.03751576 0.0317564 ]
 [0.03751576 0.04431963 0.04685151 0.04431963 0.03751576]
 [0.03965895 0.04685151 0.04952803 0.04685151 0.03965895]
 [0.03751576 0.04431963 0.04685151 0.04431963 0.03751576]
 [0.0317564  0.03751576 0.03965895 0.03751576 0.0317564 ]]
```

Gaussian Blurred Image



Aufgabe

Wendet man einen Mittelungsfiler mehrfach an so verstärkt sich der Filtereffekt.

Programmieren Sie eine Schleife, die den blur-Filter mehrfach auf ein Bild anwendet und das Ergebnis dann anzeigt Kernelgröße 7x7. Vergleichen Sie das Ergebnis mit einem einfachen blur-Filter und entsprechende angepasster Größe des Filterkerns. Sie können hier auch die OpenCV Funbktion `cv2.blur(image, kernelsize)` verwenden. (https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html). Im obigen Programmcode finden Sie einen Kommentar hierzu.

```
In [7]: #several blur steps in a row
kernel = (7,7)
noOfSteps = 51
blurSteps = image.copy()
for i in range(noOfSteps):
    blurSteps = cv2.blur(blurSteps, kernel)

plotting.displayImagePlotly(blurSteps, title=f'Blurred Image after {noOfSteps} step')
```

Blurred Image after 51 steps, Kernel (7, 7)



Beispiel Kantenfilter

Sobel Filter

Der hier vorgestellte Sobel-Ableitungsfiler ist ein Beispiel für einen Kantenfilter, der Kanten in x- und y- Richtung getrennt erkennt. Auch diesen Filter findet man implementiert in OpenCV. `cv2.Sobel()`

Hinweis: Um die Filterergebnisse etwas zu glätten, kann es sinnvoll sein, das Bild vorher mit einem Mittelungsfiler zu glätten. Probieren Sie es aus! `cv2.GaussianBlur`

Noch ein Hinweis: Da Die ableitung auch negative Werte annehmen kann wird das Filterergebnis nun mit der Funktion '`cv2.normalize`' normiert und nicht nur abgeschnitten

```
In [8]: # Image smoothing

#####
## ENTER YOU CODE HERE
#####
kerneSize = 5
sigma = 8
image_smoothed = cv2.GaussianBlur(image_f, (kerneSize,kerneSize), sigma)
plotting.displayImagePlotly(image_smoothed, title='smoothed image')
```


smoothed image



```
In [9]: # Glättung vor der Kantenfilterung
kernelSobelX = np.array([[ -3, 0, 3], [-10, 0, 10], [ -3, 0, 3]], np.float64)
sobelX = cv2.filter2D(image_smoothed, -1, kernelSobelX)

kernelSobelY = np.array([[ -3, -10, -3], [ 0, 0, 0], [ 3, 10, 3]], np.float64)
sobelY = cv2.filter2D(image_smoothed, -1, kernelSobelY)

# combine the directions (Pythagoras)
edges_sobel = np.sqrt(sobelX**2 + sobelY**2)

# scale the image back to uint8
# Normalization because of negative values
sobelX = cv2.normalize(sobelX, None, 0, 255, cv2.NORM_MINMAX).astype('uint8')
sobelY = cv2.normalize(sobelY, None, 0, 255, cv2.NORM_MINMAX).astype('uint8')
edges_sobel = cv2.normalize(edges_sobel, None, 0, 255, cv2.NORM_MINMAX).astype('ui

print('Sobel FilterkerelX')
print(kernelSobelX)

print('Sobel FilterkerelY')
print(kernelSobelY)

plotting.displayImagePlotly(sobelX, title='Sobel derivative in x-dir')
plotting.displayImagePlotly(sobelY, title='Sobel derivative in y-dir')
plotting.displayImagePlotly(edges_sobel, title='combined derivatives')
```

Sobel FilterkerelX

```
[[ -3.  0.  3.]
 [-10.  0. 10.]
 [ -3.  0.  3.]]
```

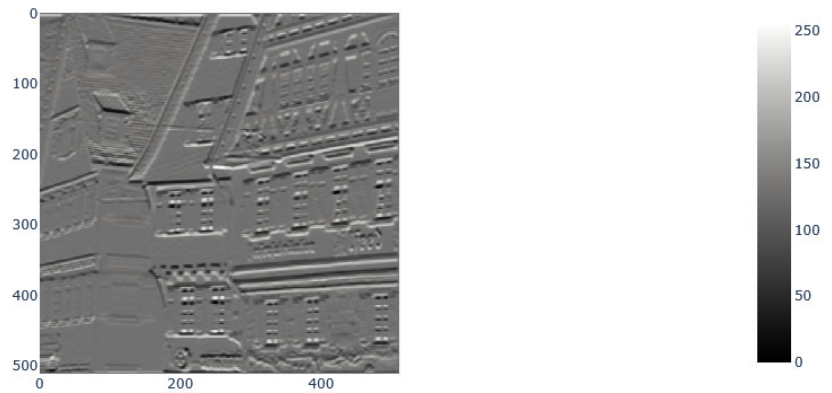
Sobel FilterkerelY

```
[[ -3. -10. -3.]
 [  0.  0.  0.]
 [  3. 10.  3.]]
```

Sobel derivative in x-dir



Sobel derivative in y-dir



combined derivatives



Laplace Filter

Der Laplace Filter ist ein Kantenfilter der die Kanten in alle Richtungen gleichzeitig sucht. Er basiert auf den 2. Ableitungen des Bildes. Daher unterscheidet sich das Ergebnis vom enen

vorgestellten Sobel Operator

Auch hier gibt es wieder eine OpenCV funktion die hier jedoch nicht aufgerufen wird
cv2.Laplacian

```
In [10]: # Laplace Filter

kernel_laplace = np.array([[0,1,0],[1,-4,1],[0,1,0]],np.float64)

# apply
laplace = cv2.filter2D(image_smoothed, -1, kernel_laplace)

# scale the image back to uint8
# Normalization because of negative values
laplace = cv2.normalize(np.abs(laplace), None, 0, 255, cv2.NORM_MINMAX).astype('uint8')

# print kernel
print('Laplace Filterkernel')
print(kernel_laplace)

# plot result
plotting.displayImagePlotly(laplace, title='Laplace Edge Detection')
```

Laplace Filterkernel

```
[[ 0.  1.  0.]
 [ 1. -4.  1.]
 [ 0.  1.  0.]
```

Laplace Edge Detection



Spezielle Filterkerne

Aufgabe

Implementieren Sie folgende Filtermatrizen und wenden sie diese auf die Bilder an. Kernel1:

$$F_1 = \frac{1}{4} \begin{bmatrix} -1 & -1 & -1 \\ -1 & 12 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Kernel2:

$$F_1 = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Wandeln Sie Ihr Ergebnis in einen `uint8` Datentyp um indem Sie die Funktion `np.clip(image, 0, 255)` verwenden

Beschreiben Sie wie sich das Bild nach Anwendung der Filter verändert hat.

Kernel1

```
In [11]: #####
          ## ENTER YOUR CODE HERE
          #####

          kernel_sharpening = 1/4*np.array([[ -1, -1, -1], [-1, 12, -1], [-1, -1, -1]], np.float64)

          # apply
          image_sharpening = cv2.filter2D(image_f, -1, kernel_sharpening)

          # scale the image back to uint8
          image_sharpening = np.clip(image_sharpening, 0, 255).astype('uint8')

          #print kernel
          print('Bilschaerfung')
          print(kernel_sharpening)

          # plot result
          plotting.displayImagePlotly(image, title='Ausgangsbild')
          plotting.displayImagePlotly(image_sharpening, title='Bilschaerfung')
```

```
Bilschaerfung
[[-0.25 -0.25 -0.25]
 [-0.25  3.   -0.25]
 [-0.25 -0.25 -0.25]]
```

Ausgangsbild



Bildscharfung



Kernel2

```
In [12]: #####
## ENTER YOUR CODE HERE
#####

kernel_relief = np.array([[ -2, -1, 0], [-1, 1, 1], [0, 1, 2]], np.float64)

# apply
image_relief = cv2.filter2D(image.astype(np.float64), -1, kernel_relief)

# clip image and back to uint8
image_relief = np.clip(image_relief, 0, 255).astype('uint8')

#print kernel
print('Bildrelief')
print(kernel_relief)

# plot result
plotting.displayImagePlotly(image, title='Ausgangsbild')
```

```
plotting.displayImagePlotly(image_relief, title='Bildrelief')
```

Bildrelief

```
[[ -2. -1.  0.]
 [ -1.  1.  1.]
 [  0.  1.  2.]]
```

Ausgangsbild



Bildrelief



Asymmetrischer Kernel

Aufgabe

Untersuchen Sie welche Auswirkungen ein asymmetrischer Kernel auf die Mittelungsfiler hat. Gehen Sie dazu folgendermaßen vor:

- glätten Sie das Bild mit Hilfe eines blur Filters `cv2.blur(img, (3,3))`. Die zweite Angabe in der Filterfunktion hier ist die Kernelgröße. in diesem Beispiel wurde eine Größe von 3x3 verwendet.
- Bringen Sie das Bild zur Anzeige
- untersuchen Sie extrem asymmetrische Kernelgrößen 3x11; 3x22, 3x51, ... und dementsprechend 55x3,

Hier eignen sich die Bilder Bricks Test und Road am besten

```
In [13]: #####  
        ## ENTER YOUR CODE HERE  
        #####  
  
        kernel = (3,101)  
        smoothed_asymm = cv2.blur(image_f, kernel)  
  
        # rescale to uint8  
        smoothed_asymm = np.clip(smoothed_asymm, 0, 255).astype('uint8')  
  
        # scale back the image to uint8  
        smoothed_asymm = np.clip(smoothed_asymm, 0, 255).astype('uint8')  
  
        # display image  
        plotting.displayImagePlotly(smoothed_asymm, title=f'Asymmetric Smoothing {kernel}')
```

Asymmetric Smoothing (3, 101)

